

Computer Vision Cheat Sheet

OpenCV · YOLO · Segmentation · Pillow · Preprocessing · Upscaling

OpenCV — Core Operations

Function / Method	Syntax	Simple explanation
import cv2	<code>import cv2</code>	Imports OpenCV library.
imread()	<code>cv2.imread('img.jpg')</code>	Reads image from disk as BGR numpy array.
imshow()	<code>cv2.imshow('win', img)</code>	Displays image in a window (desktop only).
imwrite()	<code>cv2.imwrite('out.jpg', img)</code>	Saves image to disk as JPG or PNG.
cvtColor()	<code>cv2.cvtColor(img, cv2.COLOR_BGR2RGB)</code>	Converts image between color spaces (BGR, RGB, GRAY, HSV, LAB).
COLOR_BGR2RGB	<code>cv2.COLOR_BGR2RGB</code>	Flag: OpenCV BGR order to Matplotlib RGB order.
COLOR_BGR2GRAY	<code>cv2.COLOR_BGR2GRAY</code>	Flag: converts color image to grayscale 2D array.
COLOR_BGR2HSV	<code>cv2.COLOR_BGR2HSV</code>	Flag: converts to Hue-Saturation-Value color space.
COLOR_BGR2LAB	<code>cv2.COLOR_BGR2LAB</code>	Flag: converts to LAB color space used for CLAHE.
resize()	<code>cv2.resize(img, (w, h), interpolation=...)</code>	Resizes image to given width and height.
INTER_NEAREST	<code>cv2.INTER_NEAREST</code>	Interpolation: fastest, blocky pixels, never use for products.
INTER_LINEAR	<code>cv2.INTER_LINEAR</code>	Interpolation: smooth but slightly blurry, good for thumbnails.
INTER_CUBIC	<code>cv2.INTER_CUBIC</code>	Interpolation: sharper than bilinear, good general purpose.
INTER_LANCZOS4	<code>cv2.INTER_LANCZOS4</code>	Interpolation: best OpenCV quality, use for product images.
Crop (slicing)	<code>img[y1:y2, x1:x2]</code>	Crops image using numpy array slicing. Rows first, columns second.
shape	<code>img.shape</code>	Returns (height, width, channels). Height first, then width.
dtype	<code>img.dtype</code>	Returns data type. Usually uint8 (values 0-255).

OpenCV — Drawing & Text

Function / Method	Syntax	Simple explanation
rectangle()	<code>cv2.rectangle(img, pt1, pt2, color, thickness)</code>	Draws a rectangle. pt1=(x1,y1) top-left, pt2=(x2,y2) bottom-right.
circle()	<code>cv2.circle(img, center, radius, color, thickness)</code>	Draws a circle. center=(x,y), thickness=-1 fills the circle.
line()	<code>cv2.line(img, pt1, pt2, color, thickness)</code>	Draws a straight line between two points.

Function / Method	Syntax	Simple explanation
putText()	<code>cv2.putText(img, text, org, font, scale, color, thickness)</code>	Draws text on image. org=(x,y) is bottom-left of text.
FONT_HERSHEY_SIMPLEX	<code>cv2.FONT_HERSHEY_SIMPLEX</code>	Standard font for cv2.putText(). Most commonly used.

OpenCV — Image Preprocessing

Function / Method	Syntax	Simple explanation
convertScaleAbs()	<code>cv2.convertScaleAbs(img, alpha=1.2, beta=30)</code>	Adjusts brightness (beta) and contrast (alpha). Clips to 0-255.
GaussianBlur()	<code>cv2.GaussianBlur(img, (5,5), 0)</code>	Blurs image with Gaussian kernel. Removes general noise. Kernel must be odd.
medianBlur()	<code>cv2.medianBlur(img, 5)</code>	Removes salt-and-pepper noise. Kernel size must be odd integer.
bilateralFilter()	<code>cv2.bilateralFilter(img, 9, 75, 75)</code>	Removes noise but keeps edges sharp. Best for product images.
filter2D()	<code>cv2.filter2D(img, -1, kernel)</code>	Applies custom kernel. Used for sharpening with sharpen kernel.
Sharpen kernel	<code>np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]])</code>	Custom kernel that sharpens image edges and details.
equalizeHist()	<code>cv2.equalizeHist(gray_img)</code>	Global histogram equalization. Spreads pixel values evenly.
createCLAHE()	<code>cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))</code>	Creates CLAHE object. Better than equalizeHist for product images.
clahe.apply()	<code>clahe.apply(l_channel)</code>	Applies CLAHE to L channel of LAB image. Enhances contrast locally.
split()	<code>l, a, b = cv2.split(img_lab)</code>	Splits multi-channel image into individual channel arrays.
merge()	<code>cv2.merge([l, a, b])</code>	Merges individual channel arrays back into one image.

OpenCV — Background Removal (GrabCut)

Function / Method	Syntax	Simple explanation
grabCut()	<code>cv2.grabCut(img, mask, rect, bgdModel, fgdModel, 5, cv2.GC_INIT_WITH_RECT)</code>	Separates foreground from background. rect defines foreground region.
GC_INIT_WITH_RECT	<code>cv2.GC_INIT_WITH_RECT</code>	Flag: initialise GrabCut using a rectangle region.
GC_BGD (0)	<code>mask == 0</code>	Definite background pixel in GrabCut mask.
GC_FGD (1)	<code>mask == 1</code>	Definite foreground pixel in GrabCut mask.
GC_PR_BGD (2)	<code>mask == 2</code>	Probable background pixel in GrabCut mask.
GC_PR_FGD (3)	<code>mask == 3</code>	Probable foreground pixel in GrabCut mask.
Final mask	<code>np.where((mask==2) (mask==0), 0, 1).astype('uint8')</code>	Converts GrabCut mask to binary: 1=subject, 0=background.

Function / Method	Syntax	Simple explanation
Apply mask	<code>result = img * mask[:, :, np.newaxis]</code>	Applies binary mask to image. <code>np.newaxis</code> expands mask to 3 channels.
<code>bgdModel</code> / <code>fgdModel</code>	<code>np.zeros((1, 65), np.float64)</code>	Internal model arrays required by GrabCut. Always initialise to zeros.

Computer Vision Cheat Sheet

OpenCV · YOLO · Segmentation · Pillow · Preprocessing · Upscaling

YOLO — Object Detection (Ultralytics YOLOv8)

Function / Method	Syntax	Simple explanation
import YOLO	<code>from ultralytics import YOLO</code>	Imports YOLOv8 from the ultralytics library.
Load model	<code>model = YOLO('yolov8n.pt')</code>	Loads pretrained YOLO model. Downloads automatically if not found.
Model sizes	<code>yolov8n / s / m / l / x .pt</code>	n=nano (fastest), s=small, m=medium, l=large, x=xlarge (most accurate).
Run detection	<code>results = model('image.jpg')</code>	Runs object detection. Returns a list of Results objects.
Boxes	<code>results[0].boxes</code>	Contains all detected bounding boxes for the first image.
Class ID	<code>int(box.cls[0])</code>	Integer class index of the detected object.
Class name	<code>model.names[class_id]</code>	Human-readable class name from the class index. e.g. 'cat', 'bus'.
Confidence	<code>float(box.conf[0])</code>	Detection confidence score between 0 and 1. Higher is more certain.
Bounding box	<code>x1,y1,x2,y2 = map(int, box.xyxy[0])</code>	Pixel coordinates of bounding box: top-left (x1,y1) bottom-right (x2,y2).
result.plot()	<code>annotated = results[0].plot()</code>	Draws boxes, labels and confidence scores on image. Returns BGR array.
Batch detection	<code>results = model([img1, img2, img3])</code>	Runs detection on multiple images at once.
model.names	<code>model.names</code>	Dictionary of {class_id: class_name} for all 80 COCO classes.
Confidence filter	<code>if float(box.conf[0]) >= 0.5:</code>	Filter detections by confidence threshold. Ignore low-confidence results.

YOLO — Instance Segmentation

Function / Method	Syntax	Simple explanation
Load seg model	<code>model = YOLO('yolov8n-seg.pt')</code>	Loads segmentation model. Detects objects AND generates pixel masks.
Segmentation models	<code>yolov8n/s/m/l/x-seg.pt</code>	Same size options as detection but with -seg suffix for segmentation.
Access masks	<code>results[0].masks</code>	Contains segmentation masks. None if no objects detected.
Mask data	<code>results[0].masks.data.cpu().numpy()</code>	Numpy array of shape (N, H, W). One binary mask per detected object.
Resize mask	<code>cv2.resize(mask, (w, h), interpolation=cv2.INTER_NEAREST)</code>	Resizes mask to match original image size for compositing.
Apply mask to image	<code>result = img * mask[:, :, np.newaxis]</code>	Keeps only the masked (foreground) pixels.

Function / Method	Syntax	Simple explanation
Semantic vs Instance	Semantic: same class = same color	Instance: each individual object gets unique color/mask regardless of class.
Semantic coloring	<code>semantic_mask[mask_resized] = CLASS_COLORS[class_name]</code>	Paints all pixels of same class with same color to create semantic map.
Instance coloring	<code>result.plot()</code>	Automatically assigns unique colors to each detected instance.

Pillow (PIL) — Image Creation & Compositing

Function / Method	Syntax	Simple explanation
import PIL	<code>from PIL import Image, ImageDraw, ImageFont</code>	Imports Pillow. Image for files, ImageDraw for drawing, ImageFont for fonts.
Image.new()	<code>Image.new('RGB', (800, 600), color=(41,128,185))</code>	Creates blank image. Modes: RGB, RGBA, L (grayscale). Color as RGB tuple.
Image.open()	<code>img = Image.open('product.jpg')</code>	Opens image file. Returns PIL Image object.
Image.fromarray()	<code>img = Image.fromarray(numpy_array, 'RGBA')</code>	Converts numpy array to PIL Image. Specify mode: RGB, RGBA, L.
img.resize()	<code>img.resize((width, height))</code>	Resizes image. Note: PIL uses (width, height) order, opposite of numpy.
img.convert()	<code>img.convert('RGBA')</code>	Converts image mode. RGB, RGBA, L (gray), LAB. Needed before compositing.
img.save()	<code>img.save('out.jpg', quality=95)</code>	Saves image. Format from extension. quality=95 for high quality JPG.
img.size	<code>img.size</code>	Returns (width, height) tuple. Note: opposite order from numpy shape.
img.mode	<code>img.mode</code>	Returns color mode string: 'RGB', 'RGBA', 'L' etc.
img.paste()	<code>poster.paste(product, (x, y), mask)</code>	Composites one image onto another. mask is alpha channel for transparency.
ImageDraw.Draw()	<code>draw = ImageDraw.Draw(image)</code>	Creates draw object on top of image for adding shapes and text.
draw.text()	<code>draw.text((x, y), 'text', fill=color, anchor='mm')</code>	Draws text. anchor='mm' centers text on (x,y). fill is RGB or RGBA tuple.
draw.rectangle()	<code>draw.rectangle([(x1,y1),(x2,y2)], fill=color)</code>	Draws filled or outlined rectangle. fill=-1 fills, outline draws border.
draw.ellipse()	<code>draw.ellipse([(x1,y1),(x2,y2)], fill=color)</code>	Draws ellipse or circle inside bounding box coordinates.
draw.line()	<code>draw.line([(x1,y1),(x2,y2)], fill=color, width=2)</code>	Draws a line between two points with given color and width.
Format: JPG	<code>img.save('out.jpg', quality=95)</code>	Compressed, smaller file. Loses some quality. Good for web display.
Format: PNG	<code>img.save('out.png')</code>	Lossless, larger file. Keeps every pixel. Good for print and products.

Function / Method	Syntax	Simple explanation
Format: WebP	<code>img.save('out.webp', quality=85)</code>	Modern format. Smaller than JPG, better quality. Good for mobile apps.
Format: RGBA PNG	<code>img.convert('RGBA').save('out.png')</code>	Saves with transparency channel. Use for products with removed background.

Computer Vision Cheat Sheet

OpenCV · YOLO · Segmentation · Pillow · Preprocessing · Upscaling

Image Upscaling Methods

Method	Syntax / Code	Simple explanation
INTER_NEAREST	<code>cv2.resize(img, (w,h), interpolation=cv2.INTER_NEAREST)</code>	Copies nearest pixel. Fastest but very blocky. Never use for products.
INTER_LINEAR	<code>cv2.resize(img, (w,h), interpolation=cv2.INTER_LINEAR)</code>	Bilinear: averages 4 surrounding pixels. Smooth but blurry. Thumbnails only.
INTER_CUBIC	<code>cv2.resize(img, (w,h), interpolation=cv2.INTER_CUBIC)</code>	Bicubic: uses 16 surrounding pixels. Sharper than bilinear. Good general use.
INTER_LANCZOS4	<code>cv2.resize(img, (w,h), interpolation=cv2.INTER_LANCZOS4)</code>	Best OpenCV quality. Use for product images and posters.
AI Super Res (EDSR)	<code>sr = cv2.dnn_superres.DnnSuperResImpl_create()</code>	AI-based upscaling. Understands content, reconstructs real details. Best quality.
<code>sr.readModel()</code>	<code>sr.readModel('EDSR_x4.pb')</code>	Loads pretrained EDSR model weights from .pb file.
<code>sr.setModel()</code>	<code>sr.setModel('edsr', 4)</code>	Sets algorithm and scale factor. Scale 4 means 4x upscaling.
<code>sr.upsample()</code>	<code>upscaled = sr.upsample(img_bgr)</code>	Runs AI upscaling. Input must be BGR. Returns upscaled BGR image.
When to use Lanczos	<code>if scale_needed <= 2: use Lanczos</code>	Lanczos is fast and good enough for up to 2x scaling.
When to use EDSR	<code>if scale_needed > 2: use EDSR</code>	EDSR AI gives much better results for 3x-4x scaling on product photos.

Image Representation — Fundamentals

Concept	Example / Code	Simple explanation
Grayscale image	<code>shape = (height, width)</code>	2D numpy array. Single channel. Pixel values 0 (black) to 255 (white).
Color image	<code>shape = (height, width, 3)</code>	3D numpy array. 3 channels. Each pixel has 3 values.
BGR (OpenCV)	<code>img[y, x] = [Blue, Green, Red]</code>	OpenCV reads images as BGR. Blue channel first, Red channel last.
RGB (Matplotlib)	<code>img[y, x] = [Red, Green, Blue]</code>	Matplotlib displays RGB. Red channel first, Blue channel last.
BGR to RGB fix	<code>cv2.cvtColor(img, cv2.COLOR_BGR2RGB)</code>	Always convert BGR to RGB before displaying with matplotlib.
Pixel value range	<code>0 to 255 (uint8)</code>	Raw pixel values. Divide by 255.0 to normalize for neural networks.
Normalized range	<code>0.0 to 1.0 (float32)</code>	Normalized pixel values used as input to CNN and deep learning models.
Shape order	<code>(height, width, channels)</code>	Numpy always uses rows first (height), then columns (width), then channels.

Concept	Example / Code	Simple explanation
Channel split	<code>r = img[:, :, 0], g = img[:, :, 1], b = img[:, :, 2]</code>	Extracts individual color channels using numpy slicing.
Alpha channel	<code>shape = (height, width, 4) - RGBA</code>	4th channel stores transparency. 0=transparent, 255=fully visible.
dtype uint8	<code>np.zeros((h,w,3), dtype=np.uint8)</code>	Standard image data type. Holds integer values 0-255.
dtype float32	<code>img.astype(np.float32) / 255.0</code>	Float image for neural network input. Values between 0.0 and 1.0.

AI Creative Studio — Computer Vision Pipeline

Pipeline Step	Tools Used	What it does in your project
1. Upscale	<code>cv2.resize()</code> , <code>EDSR sr.upsample()</code>	Enlarges small business product photos to minimum 512x512 for quality output.
2. Preprocess	<code>CLAHE</code> , <code>bilateralFilter</code> , <code>filter2D sharpen</code>	Enhances contrast, removes noise, sharpens product details.
3. Background remove	<code>cv2.grabCut()</code> or <code>rembg (production)</code>	Removes messy studio/desk background. Isolates product on transparent bg.
4. Detect product	<code>YOLO yolov8n.pt results[0].boxes</code>	Confirms product is visible and correctly placed. Gets exact pixel location.
5. Segment product	<code>YOLO yolov8n-seg.pt results[0].masks</code>	Gets pixel-perfect product outline for clean compositing onto poster.
6. Create poster	<code>Image.new()</code> , <code>ImageDraw</code> , <code>img.paste()</code>	Builds poster canvas, places product, adds brand color, text, price, CTA.
7. Quality validate	<code>YOLO detection + confidence threshold check</code>	Checks final poster has visible objects with confidence above 0.5 threshold.
8. Export	<code>img.save('.jpg/.png/.webp')</code>	Exports in right format: PNG for print, JPG for web, WebP for mobile.

Quick Reference — When to Use What

Task	Best Tool	Why
Read image from disk	<code>cv2.imread()</code>	OpenCV is fastest. Remember: returns BGR not RGB.
Display image	<code>plt.imshow(rgb_img)</code>	Always convert BGR to RGB first using <code>cvtColor</code> .
Resize for CNN input	<code>cv2.resize(img, (224, 224))</code>	Use <code>INTER_LANCZOS4</code> for best quality product resize.
Remove background	<code>GrabCut (practice)</code> , <code>rembg (production)</code>	GrabCut needs no install. rembg gives better edges for complex products.
Detect objects	<code>YOLO yolov8n.pt</code>	Fast, accurate, 80 COCO classes. Works on Python 3.13.
Pixel-level segmentation	<code>YOLO yolov8n-seg.pt</code>	Instance masks for precise product cropping and compositing.

Task	Best Tool	Why
Upscale small product photo	EDSR (>2x) or Lanczos (<=2x)	AI upscaling preserves product details and sharp edges.
Create poster canvas	<code>PIL Image.new()</code>	Pillow is better than OpenCV for creating and compositing layouts.
Add text to poster	<code>PIL ImageDraw.draw.text()</code>	Pillow text supports anchor positioning. <code>anchor='mm'</code> centers text.
Layer product onto poster	<code>PIL img.paste(product, pos, mask)</code>	Paste uses alpha mask for transparent background compositing.
Save final poster	<code>PIL img.save('poster.png')</code>	PNG for quality, JPG for web. Pillow handles format from extension.
Enhance dark product photo	<code>CLAHE + bilateralFilter</code>	CLAHE fixes contrast locally. Bilateral removes noise without blurring edges.